

[Open in app](#) ↗[Sign up](#)[Sign in](#)**Medium** Search

Understanding HAL (Hardware Abstraction Layer) in Android: A Developer's Guide

3 min read · Jul 14, 2025



Anil Kr Mourya

[Follow](#)

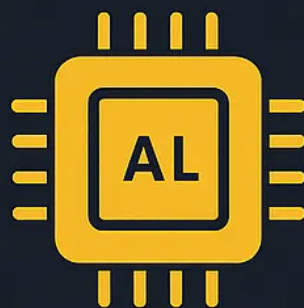


Listen



Share

HARDWARE ABSTRACTION LAYER



The HAL is an interface that allows Android to communicate with the underlying hardware.



APP



ANDROID FRAMEWORK



HAL

By abstracting the hardware, HAL makes it easier to develop apps for different devices



What Is HAL in Android?

The **Hardware Abstraction Layer (HAL)** in Android is a critical part of the OS that enables the Android framework to interact with hardware without needing to know the hardware-specific implementations.

It acts as a **bridge** between the higher-level Android services (like Camera, Bluetooth, Sensors) and the lower-level hardware drivers provided by manufacturers.

Why Do We Need HAL?

Without HAL, the Android system would need to be rewritten for every hardware variation. HAL introduces a **standardized interface** that vendors can implement, allowing Android to run on a wide range of hardware platforms **without modifying the core OS**.

How HAL Works in Android

Here's the flow:

```
App → Android Framework → HAL → Device Driver → Hardware
```

- The **App** calls an Android API (e.g., start scan, capture image).
- The **Android Framework** delegates this request to a HAL module.
- HAL communicates with the underlying **device driver**.
- The **hardware** performs the requested action.
- The **response flows back** to the app through the same layers.

HAL vs. Device Drivers

Feature	HAL	Device Driver	Layer	User space	Kernel space	Written in
	AIDL	C	Managed by Android system	Linux kernel	Purpose	Abstract hardware interaction
	Directly control hardware	Customizable by OEMs / Vendors	Chipset manufacturers			

Types of HALs in Android

- **Legacy HALs** — Pre-Android 8.0, implemented as shared libraries.
- **HIDL (HAL Interface Definition Language)** — Introduced with Android 8 (Project Treble), separates HAL implementation from the system image.
- **AIDL-based HAL** — Replacing HIDL in newer Android versions, especially from Android 11 onward for easier testing and maintenance.

Developer Use Cases for HAL

- Integrating custom hardware like **RFID readers**, **barcode scanners**, or **UHF modules**.
- Building **AOSP-based custom ROMs** with vendor-specific hardware features.
- Creating apps for **industrial devices** that need to directly access hardware through vendor HAL SDKs.
- Using **vendor-provided libraries** that internally communicate with HAL via AIDL or system services.

Tools & Debugging

- `logcat` : View HAL service logs
- `adb shell dumpsys` : Inspect service bindings
- `adb shell getprop` : Check HAL-related properties
- HIDL/AIDL logs for interface binding issues

⚠ Common Challenges

- Limited documentation from vendors
- Version compatibility issues between HAL and Android OS
- Permissions and service binding problems
- Difficult to emulate HAL in standard Android emulators (real device testing is required)

🔧 HAL in Modern Android (Treble Architecture)

Project Treble (Android 8+) modularized the HAL layer, allowing device manufacturers to update HAL modules **without touching the Android system image**.

✅ This made system updates faster, easier, and more secure.

Modern Android uses **AIDL** over **HIDL** for newer HALs, improving developer experience with better tooling and testability.

▲ Final Thoughts

HAL may be hidden under the surface for most Android developers, but it's a **powerful layer** for those working with custom hardware, AOSP builds, or enterprise apps on specialized devices.

Get Anil Kr Mourya's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Understanding HAL helps you:

- Work closely with hardware vendors
- Troubleshoot low-level hardware behavior
- Create tightly integrated Android solutions for industrial, logistics, and IoT applications

Whether you're an embedded Android developer or building apps for specialized hardware, **HAL is the key to unlocking full hardware potential** on Android.

 **Follow for more**

I write about Android system integration, hardware SDKs, and scalable mobile architecture. Feel free to connect or share your questions in the comments!

Hal

Android

Android App Development

Java

Hardware



Follow

Written by Anil Kr Mourya

89 followers · 134 following

Unlocking the Power of Developers : Your Imagination, Our Innovation. LinkedIn Profile <https://www.linkedin.com/in/anil-kr-maurya-351a48283/>

No responses yet



Write a response

What are your thoughts?

